Multilingual advanced search in Django without cloud

Thomas Aglassinger, 2026-04-19

About me

Thomas Aglassinger, Software developer

- 25+ years of industry experience in various sectors (banking, e-commerce, health, ...)
- self- and part-time employed
- founder of Siisurit: https://siisurit.com/
- casual open source developer: https://roskakori.at/
- organizer of Python Graz user group: https://pygraz.org/

Motivation for this approach

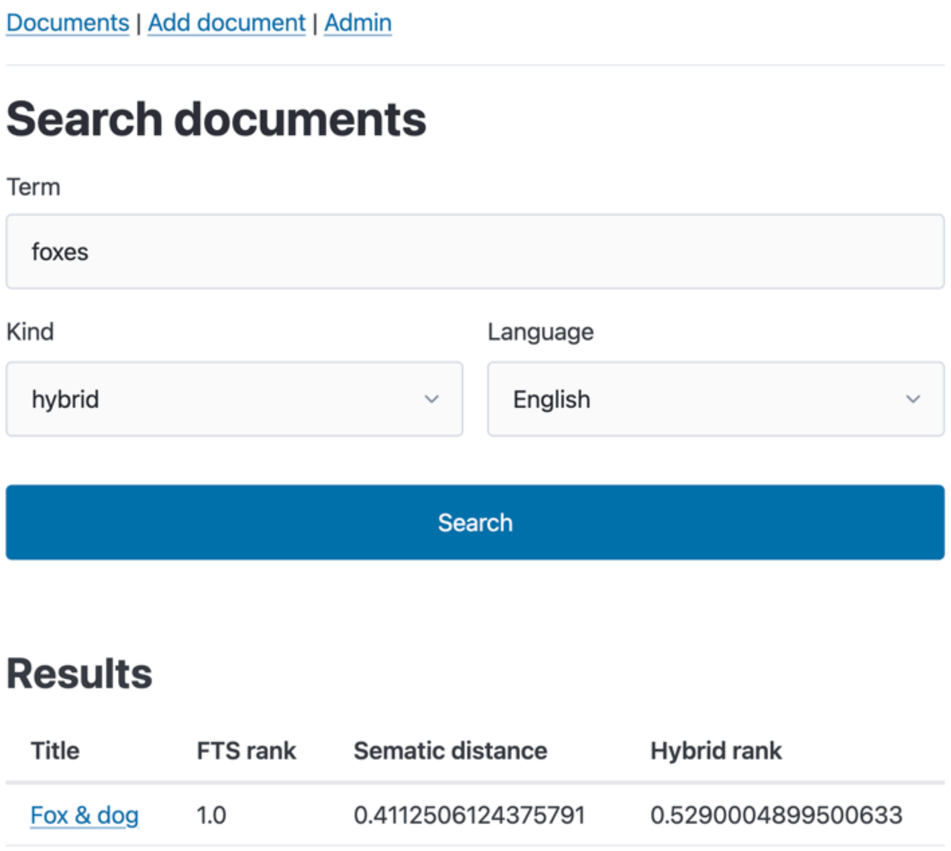
- Little code → easier to understand, fewer bugs
- Few dependencies
  - less breakage
  - better security (supply chain attacks)
- No cloud
  - data sovereignty
  - independencs (price hikes, enshittification)

Agenda

- Different ways to search in Django+Postgres
- Demo app to showcase the approaches (MIT license)
- Beyond search with Postgres
- Pillars of good search results

Demo app

Demo app



- Try different kind of searches
- Add documents in Django admin site
- Prepare semantic search with management command: python manage.py update_all_document_sematic_vectors

Demo app limitations

- Insecure default settings → run only locally
- English only, no localization
- Basic tests
- Timezone UTC only
- No pagination, only top results
- Simple design (pico.css)

Simple document model

- iso_language (en, de, it, fr, ...)
  - and "other"
- title
- content

```
class Document(models.Model):
    iso_language = models.CharField(
        choices=IsoLanguage.choices,
        default=IsoLanguage.OTHER,
        max_length=2,
        verbose_name="ISO language",
    )
    title = models.CharField(
        blank=True,
        default="",
        max_length=MAX_TITLE_LENGTH,
    )
    content = models.TextField(
        blank=True,
        default="",
    )
```

Example document

...in Django admin site

Add document

ISO language:	English
Title:	Fox & dog
Content:	The quick brown fox jumps over the lazy white dog

Plain search

Plain search

```
documents_found = (
    Document.objects.only("iso_language", "title", "content")
    .filter(iso_language=iso_language)
    .filter(
        Q(title__icontains=term) | Q(content__icontains=term),
    )
    .order_by("-id")
)[:MAX_SEARCH_RESULT_COUNT]
```

Plain search example

Documents | Add document | Admin

Search documents

Term

fox

Kind

plain

Language

English

Search

Results

Title	FTS rank	Sematic distance	Hybrid rank
Fox & dog	None	None	None

Plain search example

Documents | Add document | Admin

Search documents

Term

foxes

Kind

plain

Language

English

Search

Results

Title	FTS rank	Sematic distance	Hybrid rank

Full-text search (FTS)

Tokenization

- Split text into words
- Each word is assigned an int
- Text turns into list of int
- + Faster comparison
- + Less storage

Stopwords

- Remove words that have little to no meaning
- Examples:
  - English: The, a
  - German: der, einer

Stemming

- Remove ending for otherwise similar words
- Only heuristic, not proper grammar → still covers many common cases
- Example from English:
  - walked -> walk
  - foxes -> fox

Django VectorField

To store the transformed text as list of int

```
class Document(models.Model):
    title: str = models.CharField(blank=True, default="", max_length=MAX_TITLE_LENGTH)
    fts_vector = SearchVectorField(
        default=None,
        editable=False,
        null=True,
        verbose_name="vector for full-text search"
    )
```

SearchVector

To store in VectorField

- Is derived from other fields in model
- Can only be stored after model has been saved
- How to do this?
  - set and save(update_fields=["search_vector"]
- When to do this?
  - After create()
  - Every night
  - With database trigger

SearchVector

To store in VectorField

- Is derived from other fields in model
- Can only be stored after model has been saved
- How to do this?
  - set and save(update_fields=["search_vector"]
- When to do this?
  - After create()
  - Every night
  - With database trigger

Cumbersome!

SearchVector as generated field

Since Django 5.0

```
fts_vector = models.GeneratedField(
    expression=SearchVector("title"),
    output_field=SearchVectorField(),
    db_persist=True,
    verbose_name="vector for full-text search",
)
```

SearchVector with selected language

```
fts_vector = models.GeneratedField(
    expression=SearchVector("title", config="english"),
    output_field=SearchVectorField(),
    db_persist=True,
    verbose_name="vector for full-text search",
)
```

FTS configurations supported by Postgres

```
% echo "select cfgname from pg_ts_config order by 1;" | uv run python manage.py dbshell
  cfgname
------------
 arabic
 armenian
 basque
 catalan
 danish
 dutch
 english
 estonian
 finnish
 french
 german
 greek
 hindi
 hungarian
 indonesian
 irish
 italian
 lithuanian
 nepali
 norwegian
 portuguese
 romanian
 russian
 serbian
 simple
 spanish
 swedish
 tamil
 turkish
 yiddish
(30 rows)
```

SearchVector with language in other field

Case ... When ... default

```
fts_vector = models.GeneratedField(
    expression=Case(
        When(iso_language=IsoLanguage.ENGLISH, then=SearchVector("title", config="english")),
        When(iso_language=IsoLanguage.FRENCH, then=SearchVector("title", config="french")),
        When(iso_language=IsoLanguage.GERMAN, then=SearchVector("title", config="german")),
        When(iso_language=IsoLanguage.ITALIAN, then=SearchVector("title", config="italian")),
        default=SearchVector("title", weight="A", config="simple"),
    ),
    output_field=SearchVectorField(),
    db_persist=True,
    verbose_name="vector for full-text search",
)
```

Language mappings

```python
class IsoLanguage(TextChoices):
    """
    ISO 639-1 language codes and their
    human-readable names.
    """

    ENGLISH = "en", "English"
    FRENCH = "fr", "French"
    GERMAN = "de", "German"
    ITALIAN = "it", "Italian"
    OTHER = "xx", "Other"
```

```python
"""
Map ISO 639-1 language to Postgres
language configuration.
"""
ISO_LANGUAGE_TO_TS_CONFIG_MAP = {
    IsoLanguage.ENGLISH: "english",
    IsoLanguage.FRENCH: "french",
    IsoLanguage.GERMAN: "german",
    IsoLanguage.ITALIAN: "italian",
    IsoLanguage.OTHER: "simple",
}
```

Generated field with language mapping

```
fts_vector = models.GeneratedField(
    expression=Case(
        *[
            When(iso_language=iso_language, then=SearchVector("title", config=config))
            for iso_language, config in ISO_LANGUAGE_TO_TS_CONFIG_MAP.items()
        ],
        default=SearchVector("title", weight="A", config="simple"),
    ),
    output_field=SearchVectorField(),
    db_persist=True,
    verbose_name="vector for full-text search",
)
```

Search multiple fields: title and content

```
fts_vector = models.GeneratedField(
    expression=Case(
        *[
            When(
                iso_language=iso_language,
                then=(
                    SearchVector("title", config=config)
                    + SearchVector("content", config=config)
                ),
            )
            for iso_language, config in ISO_LANGUAGE_TO_TS_CONFIG_MAP.items()
        ],
        default=(
            SearchVector("title", weight="A", config="simple")
            + SearchVector("content", weight="B", config="simple")
        ),
    ),
```

Weight: Title is more important than content

```
fts_vector = models.GeneratedField(
    expression=Case(
        *[
            When(
                iso_language=iso_language,
                then=(
                    SearchVector("title", weight="A", config=config)
                    + SearchVector("content", weight="B", config=config)
                ),
            )
            for iso_language, config in ISO_LANGUAGE_TO_TS_CONFIG_MAP.items()
        ],
        default=(
            SearchVector("title", weight="A", config="simple")
            + SearchVector("content", weight="B", config="simple")
        ),
    ),
```

How to search a term and order by rank

```
search_query = SearchQuery(term, search_type="raw", config=config)
search_rank = SearchRank(F("fts_vector"), search_query)

documents_found = (
    Document.objects.only("iso_language", "title", "content")
    .annotate(rank=search_rank)
    .filter(
        iso_language=iso_language,
        fts_vector=search_query,
        rank__gt=0.0)
    .order_by("-rank")  # Yield most relevant entries first.
)[:MAX_SEARCH_RESULT_COUNT]
```

Search type

- Different search types: https://docs.djangoproject.com/en/6.0/ref/contrib/postgres/search/#searchquery
- web_search: seemingly useful, but connects multiple terms with AND = all must be found in document → most people expect OR = the more the better, but one is enough for the document to be found
- raw: must sanitize user input to avoid SQL injection
  - Find any word of: 'some' | 'other' | 'this' | 'that'

Faster with index

```
class Meta:
    indexes = [
        # Index for full-text search.
        GinIndex(
            fields=["fts_vector"],
            name="%(app_label)s_%(class)s_fts_vector_idx",
        ),
    ]
```

Mixing languages in FTS

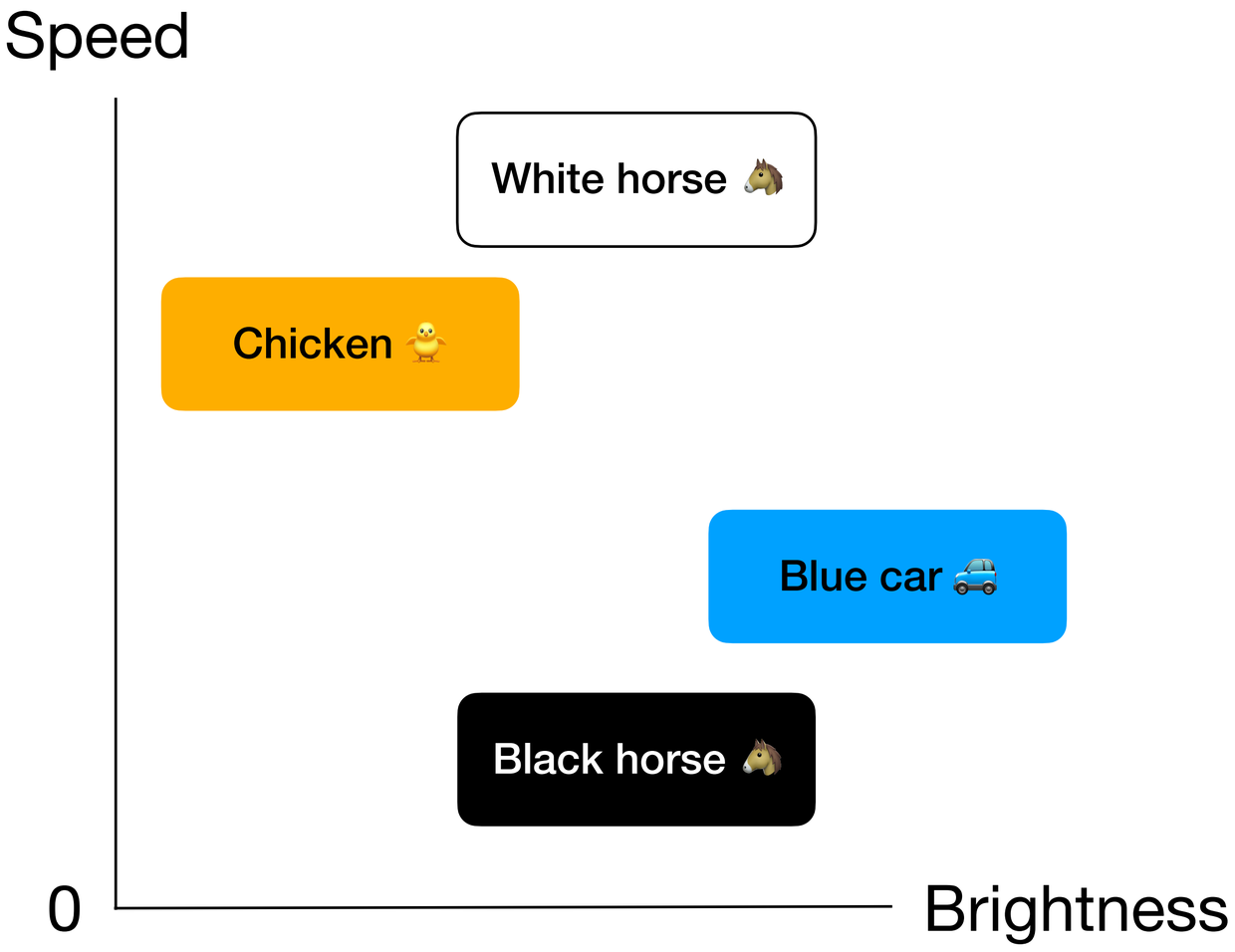
- Some words result in the same token:
  "Katze" (cat) = "katz" in German and English
- Some words result in different tokens and consequently won't be found:
  "Häuser" (houses) = "haus" in German and "häuser" in English

Semantic search

Semantic vector

- Converts text to list of float between 0.0 and 1.0
- Each position in the list has a meaning
- Compute distance between search term and document
- Smaller distance = similar meaning

Simple vector example: 2 dimensions



	Speed	Brightness
White horse	0.6	1.0
Chicken	0.1	0.8
Blue car	1.0	0.4
Black horse	0.6	0.1

Ollama embedding models

- Browse: https://ollama.com/search?c=embedding
- models supporting many languages, for example, qwen3-embedding (100+)
- models specialized on one or few languages, for example, for German: jina/jina-embeddings-v2-base-de

Ollama REST API

- Visit https://docs.ollama.com/api/introduction
- Interact with:
  1. via ollama Python client: https://pypi.org/project/ollama/
  2. directly with HTTP requests
     - minimal code
     - better understanding
     - no supply chain attacks

Pulling a model to your local Ollama server

https://docs.ollama.com/api/pull

cURL Default

```
curl http://localhost:11434/api/pull -d '{
  "model": "gemma3"
}'
```

200

```
{
  "status": "success"
}
```

Computing embeddings for a text

https://docs.ollama.com/api/embed

cURL Default

```
curl http://localhost:11434/api/embed -d '{
  "model": "embeddinggemma",
  "input": "Why is the sky blue?"
}'
```

- Returns vector = list[float]
- Can be stored in Postgres vector field

200


```
{
  "model": "embeddinggemma",
  "embeddings": [
    [
      0.010071029,
      -0.0017594862,
      0.05007221,
      0.04692972,
      0.054916814,
      0.008599704,
      0.105441414,
      -0.025878139,
      0.12958129,
      0.031952348
    ]
  ],
  "total_duration": 14143917,
  "load_duration": 1019500,
  "prompt_eval_count": 8
}
```

Mixing languages in semantic search

- If different languages use the same embedding model: same search result
- If different languages use different embedding models:
  - For example:
    - English: "qwen3-embedding"
    - German: "jina-embeddings-v2-base-de"
  - Different dimensions in the vector might have different meanings. Fictional example
    - English: color, size, speed = 0.3, 0.7, 0.4
    - German: size, color, price = 0.3, 0.7, 0.4
    - Same numbers, completely different meaning

Hybrid search

Principle

- Run query to obtain both FTS and semantic search results
- Combine them into a single result
- Various approaches for the latter

	FTS rank	Semantic distance	Combined
Document 1	0.5	0.8	1.3
Document 2	0.6		0.6
Document 3		0.4	0.4
Document 4	0.3	0.7	1.0

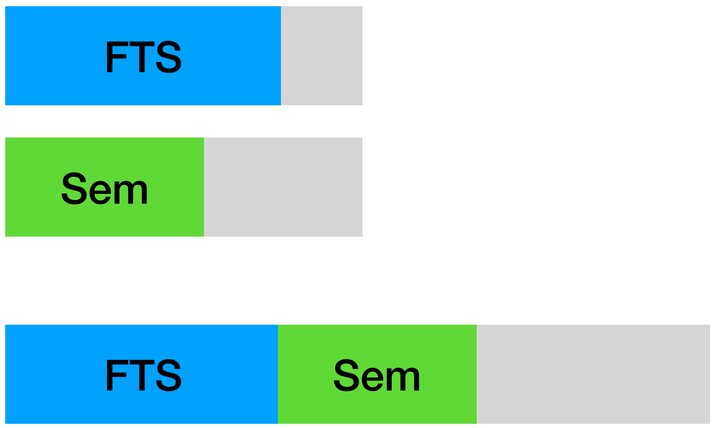
Normalized sum

Problem: FTS rank technically can be any range

- Normalize FTS rank to a value between 0.0 and 1.0.
- Semantic cosine distance is already between 0.0 and 1.0 → done

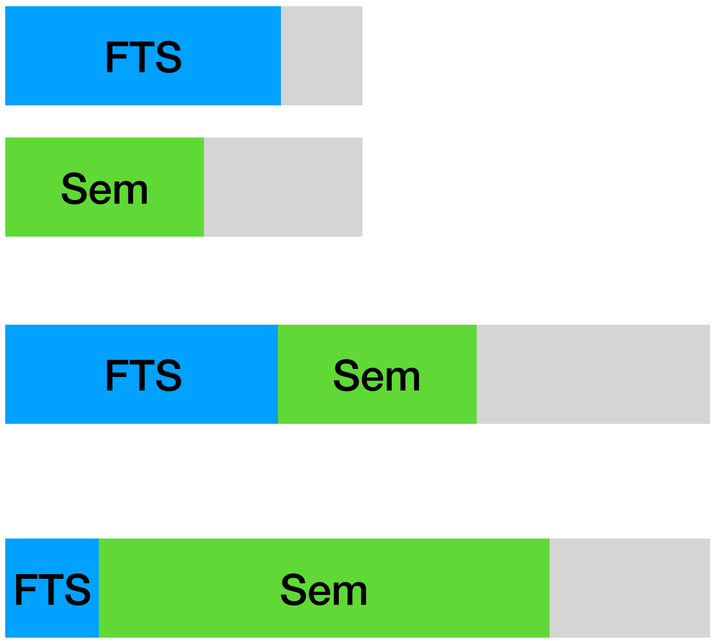
Normalized sum

- Normalize FTS rank to a value between 0.0 and 1.0.
- Semantic cosine distance ist already between 0.0 and 1.0 → done
- Add FTS rank to semantic distance → value between 0.0 and 2.0



Weighted normalized sum

- Normalize FTS rank to a value between 0.0 and 1.0.
- Semantic cosine distance ist already between 0.0 and 1.0 → done
- Add FTS rank to semantic distance → value between 0.0 and 2.0
- Use different weights, for example:
  - FTS rank * 0.2 + semantic distance * 0.8



Reciprocal Rank Fusion (RRF)

- hybrid = 1 / (FTS + k) + 1 (Sem + k) with for example k= 60
- weighted variant
- Bruch, Gai, and Ingber: An Analysis of Fusion Functions for Hybrid Retrieval, https://arxiv.org/pdf/2210.11934

Search beyond PostgreSQL and pg_vector

tf-idf

- tf = term frequency: if a document contains the search term more often, it ranks higher
- idf = indirect document frequency: common terms that occur in more documents are less valuable → similar to stopword
- tf: term occurs 1000 times -> document is 1000 times higher ranked
- BM25: logarithmic tf

pg_textsearch

BM25 for Postgres

- Adds BM25 to Postgres
- Article: From ts_rank to BM25. Introducing pg_textsearch: True BM25 Ranking and Hybrid Retrieval Inside Postgres
- GitHub project: https://github.com/timescale/pg_textsearch
- Version 1.0 released 2026-03-27 (3 weeks ago)
- No Django integration yet, but looks promising

```
SELECT
  id,
  title,
  content <@> to_bm25query('database performance', 'articles_content_idx') AS rank
FROM articles
ORDER BY rank LIMIT 5;
```

Solr debug query

Understanding how a rank came to be

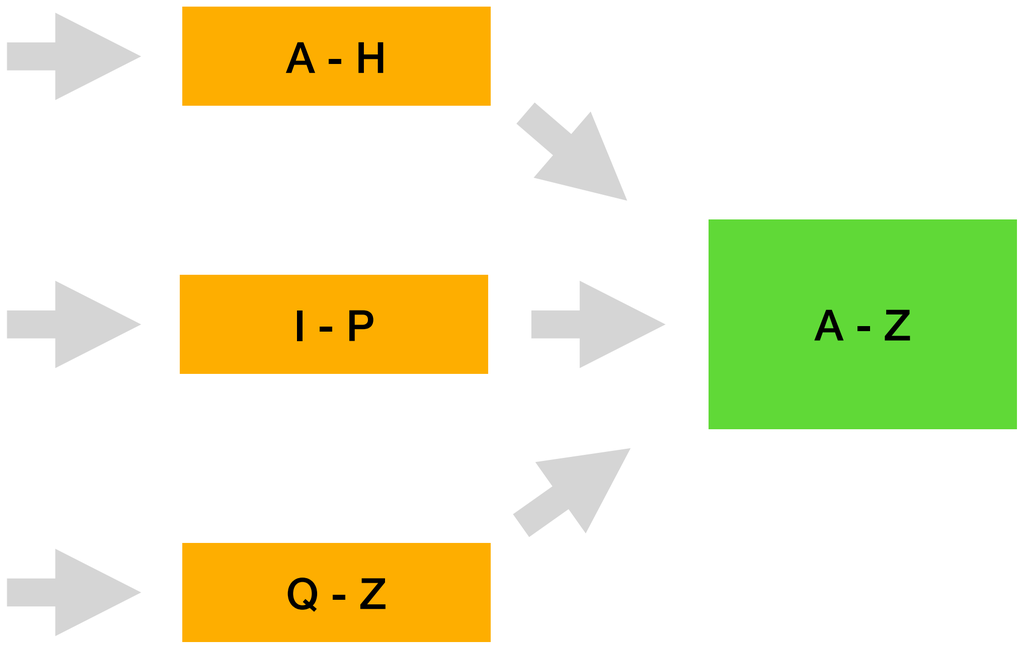
Article: Solr debugQuery

```
3.8011298 = weight(title_txt_en:blue in 5842) [SchemaSimilarity], result of:
  3.8011298 = score(freq=1.0), computed as boost * idf * tf from:
    6.1900153 = idf, computed as log(1 + (N - n + 0.5) / (n + 0.5)) from:
        20 = n, number of documents containing term
            10000 = N, total number of documents with field
              0.61407435 = tf, computed as freq / (freq + k1 * (1 - b + b * dl / avgdl)) from:
                1.0 = freq, occurrences of term within document
                  1.2 = k1, term saturation parameter
                    0.75 = b, length normalization parameter
                      2.0 = dl, length of field
                        5.48 = avgdl, average length of field
```

Sharding

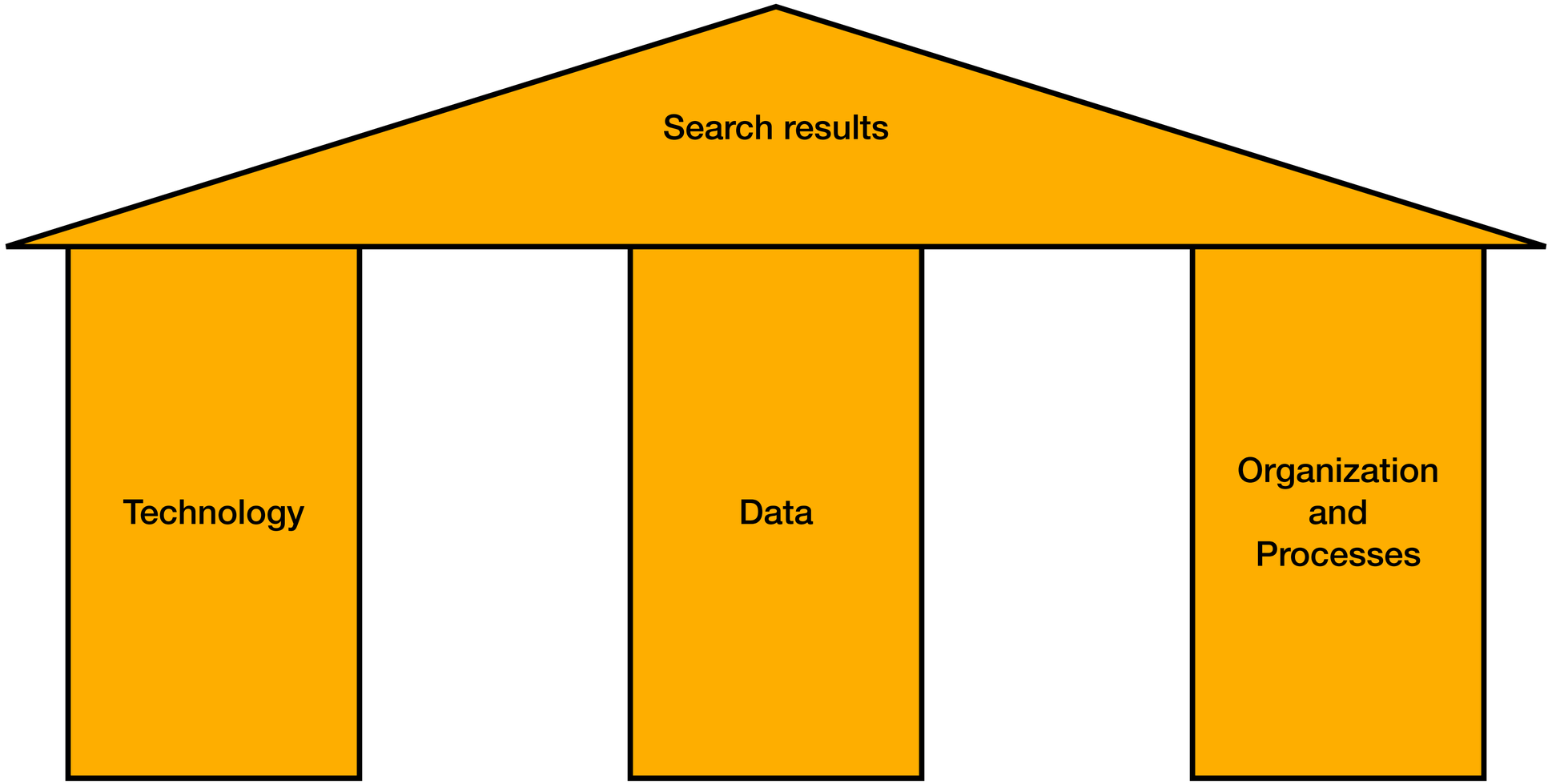
Quickly search millions and billions of documents

- Split all documents into different "shards"
  - shard = kind of separate database
  - can be on a separate server
- Search all shards in parallel
- Combine the results into a single one
- https://solr.apache.org/guide/solr/latest/deployment-guide/solrcloud-shards-indexing.html



Pillars of good search results

Pillars of good search results



Story time

- Inconsistent product descriptions in e-commerce
- Track actual searches
- "Learn to rank"

Conclusion

Conclusion

- Django has multiple ways to search text
- Pick the one that suits your situation best in terms of development effort, resource need, architectural complexity, performanceantic
- Missing features:
  - tf-idf / BM25 → out-of-the-box ranking
  - Sharding → search really large datasets (millions, billions)
- Solid, stable solution for many search problems